

Deep Learning

Vazgen Mikayelyan



1 Autoencoders

What is Unsupervised Learning?

What is Unsupervised Learning?

Definition 1

Unsupervised learning is a machine learning technique that finds and analyzes hidden patterns in unlabeled data.

What is Unsupervised Learning?

Definition 1

Unsupervised learning is a machine learning technique that finds and analyzes hidden patterns in unlabeled data.

Examples?

Autoencoders

Autoencoders are neural networks that aims to copy their inputs to their outputs. They work by compressing the input into a latent-space representation, and then reconstructing the output from this representation.

Autoencoders are neural networks that aims to copy their inputs to their outputs. They work by compressing the input into a latent-space representation, and then reconstructing the output from this representation. This kind of network is composed of two parts

- **Encoder:**

Autoencoders are neural networks that aims to copy their inputs to their outputs. They work by compressing the input into a latent-space representation, and then reconstructing the output from this representation. This kind of network is composed of two parts

- **Encoder:**

This is the part of the network that compresses the input into a latent-space representation.

Autoencoders are neural networks that aims to copy their inputs to their outputs. They work by compressing the input into a latent-space representation, and then reconstructing the output from this representation. This kind of network is composed of two parts

- **Encoder:**

This is the part of the network that compresses the input into a latent-space representation.

- **Decoder:**

Autoencoders are neural networks that aims to copy their inputs to their outputs. They work by compressing the input into a latent-space representation, and then reconstructing the output from this representation. This kind of network is composed of two parts

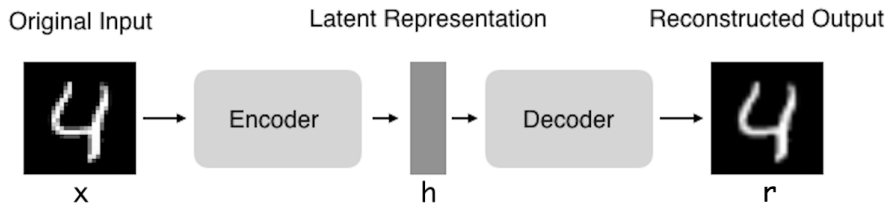
- **Encoder:**

This is the part of the network that compresses the input into a latent-space representation.

- **Decoder:**

This part aims to reconstruct the input from the latent space representation.

Autoencoders



What are autoencoders used for?

- Anomaly/Outlier detection.

What are autoencoders used for?

- Anomaly/Outlier detection.
- Dimensionality reduction.

What are autoencoders used for?

- Anomaly/Outlier detection.
- Dimensionality reduction.
- Data denoising.

What are autoencoders used for?

- Anomaly/Outlier detection.
- Dimensionality reduction.
- Data denoising.
- In a lot of different tasks.

Types of Autoencoders

- Vanilla Autoencoders

Types of Autoencoders

- Vanilla Autoencoders
- Multilayer/Deep Autoencoders

Types of Autoencoders

- Vanilla Autoencoders
- Multilayer/Deep Autoencoders
- Convolutional Autoencoders

Types of Autoencoders

- Vanilla Autoencoders
- Multilayer/Deep Autoencoders
- Convolutional Autoencoders
- Contractive Autoencoders

Types of Autoencoders

- Vanilla Autoencoders
- Multilayer/Deep Autoencoders
- Convolutional Autoencoders
- Contractive Autoencoders
- Regularized Autoencoders

Types of Autoencoders

- Vanilla Autoencoders
- Multilayer/Deep Autoencoders
- Convolutional Autoencoders
- Contractive Autoencoders
- Regularized Autoencoders
 - Sparse Autoencoders

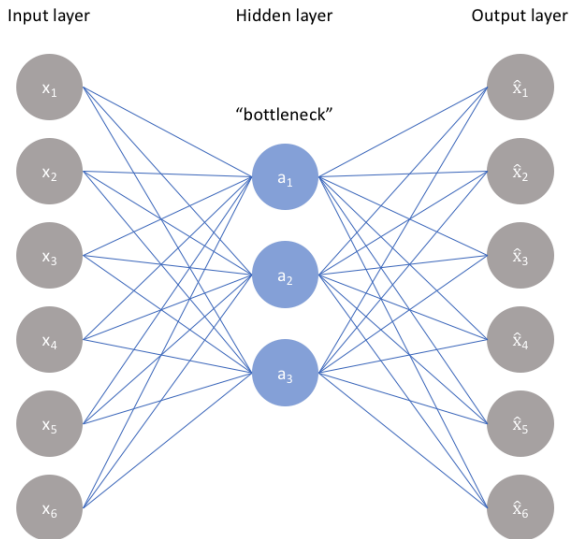
Types of Autoencoders

- Vanilla Autoencoders
- Multilayer/Deep Autoencoders
- Convolutional Autoencoders
- Contractive Autoencoders
- Regularized Autoencoders
 - Sparse Autoencoders
 - Denoising Autoencoders

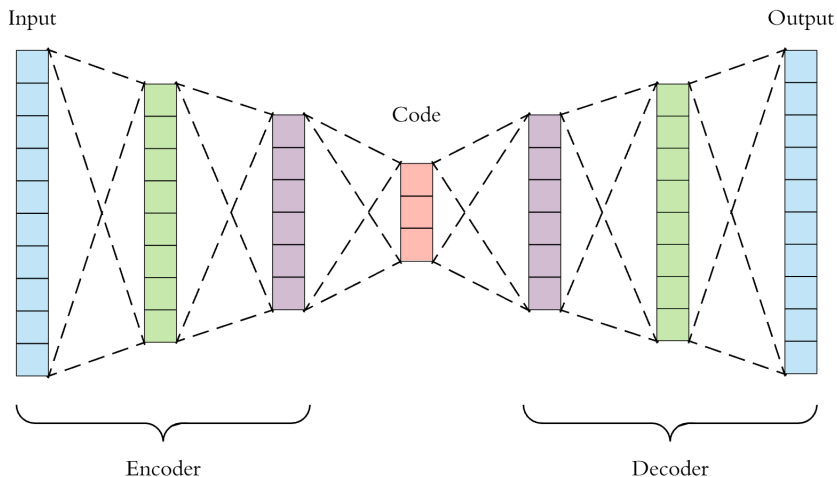
Types of Autoencoders

- Vanilla Autoencoders
- Multilayer/Deep Autoencoders
- Convolutional Autoencoders
- Contractive Autoencoders
- Regularized Autoencoders
 - Sparse Autoencoders
 - Denoising Autoencoders
- Variational Autoencoders

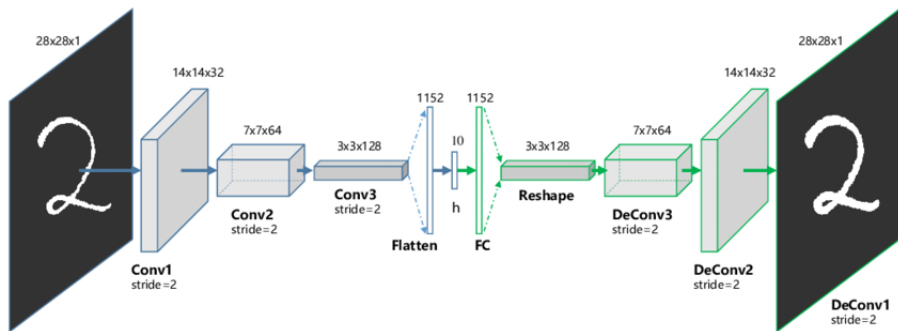
Vanilla Autoencoders



Multilayer/Deep Autoencoders



Convolutional Autoencoders



- A contractive autoencoder makes this encoding less sensitive to small variations in its training dataset.

Contractive Autoencoders

- A contractive autoencoder makes this encoding less sensitive to small variations in its training dataset.
- This is accomplished by adding a regularizer, or penalty term, to whatever cost or objective function the algorithm is trying to minimize.

Contractive Autoencoders

- A contractive autoencoder makes this encoding less sensitive to small variations in its training dataset.
- This is accomplished by adding a regularizer, or penalty term, to whatever cost or objective function the algorithm is trying to minimize.
- The end result is to reduce the learned representation's sensitivity towards the training input.

Contractive Autoencoders

Let f is our encoder, g is the decoder and D is our training dataset. In the previous cases we minimize this kind of loss function:

$$\sum_{x \in D} L(x, g(f(x))).$$

Contractive Autoencoders

Let f is our encoder, g is the decoder and D is our training dataset. In the previous cases we minimize this kind of loss function:

$$\sum_{x \in D} L(x, g(f(x))).$$

In the case of contractive autoencoders we will minimize this one

$$\sum_{x \in D} \left(L(x, g(f(x))) + \lambda \|J_f(x)\|_F^2 \right),$$

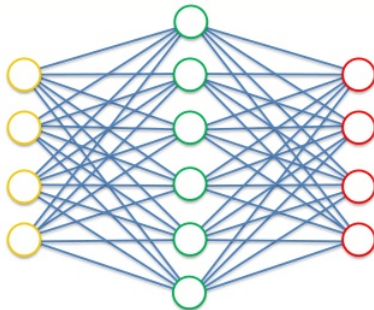
where the added summand is the square of Frobenius norm of the following Jacobian matrix:

$$[J_f(x)]_{i,j} = \frac{\partial f_j(x)}{\partial x_i}$$

i.e.

$$\|J_f(x)\|_F^2 = \sum_{i,j} \left(\frac{\partial f_j(x)}{\partial x_i} \right)^2.$$

Sparse Autoencoders



Sparse Autoencoders

- Sparse autoencoders have hidden nodes greater than input nodes. They can still discover important features from the data.

Sparse Autoencoders

- Sparse autoencoders have hidden nodes greater than input nodes. They can still discover important features from the data.
- Sparsity penalty is introduced on the hidden layer. This is to prevent output layer copy input data. This prevents overfitting.

Sparse Autoencoders

Let f is our encoder, g is the decoder and D is our training dataset, which has n samples. Denote

$$\rho_j = \frac{1}{n} \sum_{x \in D} f_j(x).$$

Sparse Autoencoders

Let f is our encoder, g is the decoder and D is our training dataset, which has n samples. Denote

$$\rho_j = \frac{1}{n} \sum_{x \in D} f_j(x).$$

We would like to (approximately) enforce the constraint $\rho_j = \rho$, where ρ is a sparsity parameter, typically a small value close to zero (say $\rho = 0.05$).

Sparse Autoencoders

Let f is our encoder, g is the decoder and D is our training dataset, which has n samples. Denote

$$\rho_j = \frac{1}{n} \sum_{x \in D} f_j(x).$$

We would like to (approximately) enforce the constraint $\rho_j = \rho$, where ρ is a sparsity parameter, typically a small value close to zero (say $\rho = 0.05$). To achieve this we will minimize the following loss function

$$\sum_{x \in D} L(x, g(f(x))) + \lambda \sum_j KL(\rho || \rho_j),$$

Sparse Autoencoders

Let f is our encoder, g is the decoder and D is our training dataset, which has n samples. Denote

$$\rho_j = \frac{1}{n} \sum_{x \in D} f_j(x).$$

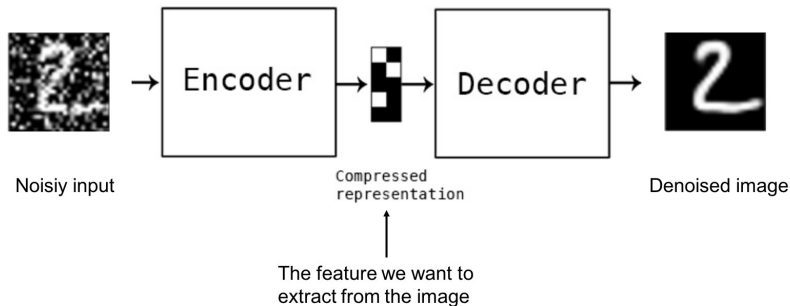
We would like to (approximately) enforce the constraint $\rho_j = \rho$, where ρ is a sparsity parameter, typically a small value close to zero (say $\rho = 0.05$). To achieve this we will minimize the following loss function

$$\sum_{x \in D} L(x, g(f(x))) + \lambda \sum_j KL(\rho || \rho_j),$$

where

$$KL(\rho || \rho_j) = -\rho \log \frac{\rho_j}{\rho} - (1 - \rho) \log \frac{1 - \rho_j}{1 - \rho}.$$

Denoising Autoencoders



Denoising Autoencoders

- Denoising autoencoders create a corrupted copy of the input by introducing some noise.

Denoising Autoencoders

- Denoising autoencoders create a corrupted copy of the input by introducing some noise.
- This helps to avoid the autoencoders to copy the input to the output without learning features about the data.

Denoising Autoencoders

- Denoising autoencoders create a corrupted copy of the input by introducing some noise.
- This helps to avoid the autoencoders to copy the input to the output without learning features about the data.
- The model learns a vector field for mapping the input data towards a lower dimensional manifold which describes the natural data to cancel out the added noise.

- Can we generate content with autoencoders?

- Can we generate content with autoencoders?
- Variational autoencoder can be defined as being an autoencoder whose training is regularised to avoid overfitting and ensure that the latent space has good properties that enable generative process.

- Can we generate content with autoencoders?
- Variational autoencoder can be defined as being an autoencoder whose training is regularised to avoid overfitting and ensure that the latent space has good properties that enable generative process.
- Instead of encoding an input as a single point, we encode it as a distribution over the latent space.

Variational Autoencoders

- Denote distribution of latent representations by $p(z)$.

Variational Autoencoders

- Denote distribution of latent representations by $p(z)$.
- Denote distribution of ideal encoder by $p(z|x)$.

Variational Autoencoders

- Denote distribution of latent representations by $p(z)$.
- Denote distribution of ideal encoder by $p(z|x)$.
- We want to find a neural network $q_w(z|x)$ such that

$$q_w(z|x) \approx p(z|x).$$

Variational Autoencoders

- Denote distribution of latent representations by $p(z)$.
- Denote distribution of ideal encoder by $p(z|x)$.
- We want to find a neural network $q_w(z|x)$ such that

$$q_w(z|x) \approx p(z|x).$$

- Let minimize the function

$$L(w) = KL(q_w(z|x) || p(z|x)).$$

Variational Autoencoders

Note that

$$KL(q_w(z|x) || p(z|x)) = \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right]$$

Variational Autoencoders

Note that

$$\begin{aligned} KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\ &= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \end{aligned}$$

Variational Autoencoders

Note that

$$\begin{aligned} KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\ &= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\ &= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \end{aligned}$$

Variational Autoencoders

Note that

$$\begin{aligned} KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\ &= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\ &= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \\ &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z)} \right] - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \mathbb{E}_{q_w(z|x)} [\log p(x)] \end{aligned}$$

Variational Autoencoders

Note that

$$\begin{aligned}KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \\&= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z)} \right] - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \mathbb{E}_{q_w(z|x)} [\log p(x)] \\&= KL(q_w(z|x) || p(z)) - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \log p(x).\end{aligned}$$

Variational Autoencoders

Note that

$$\begin{aligned}KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \\&= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z)} \right] - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \mathbb{E}_{q_w(z|x)} [\log p(x)] \\&= KL(q_w(z|x) || p(z)) - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \log p(x).\end{aligned}$$

So we have to model the distribution $p(x|z)$ too, which will be our decoder:

Variational Autoencoders

Note that

$$\begin{aligned}KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \\&= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z)} \right] - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \mathbb{E}_{q_w(z|x)} [\log p(x)] \\&= KL(q_w(z|x) || p(z)) - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \log p(x).\end{aligned}$$

So we have to model the distribution $p(x|z)$ too, which will be our decoder:

$$\operatorname{argmin}_{w, w'} (KL(q_w(z|x) || p(z)) + \mathbb{E}_{q_w(z|x)} [\|x - f_{w'}(z)\|^2])$$

Problem: how to evaluate the term $KL(q_w(z|x) || p(z))$?

Problem: how to evaluate the term $KL(q_w(z|x) || p(z))$?

Solution: we will assume that $q_w = \mathcal{N}(\mu, \Sigma)$ and $p = \mathcal{N}(0, I)$.

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,
- third, the sampled point is decoded and the reconstruction error can be computed,

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,
- third, the sampled point is decoded and the reconstruction error can be computed,
- finally, the reconstruction error is backpropagated through the network.

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,
- third, the sampled point is decoded and the reconstruction error can be computed,
- finally, the reconstruction error is backpropagated through the network.

Assumptions

- In practice, the encoded distributions are chosen to be normal so that the encoder can be trained to return the mean and the covariance matrix that describe these Gaussians.

Variational Autoencoders

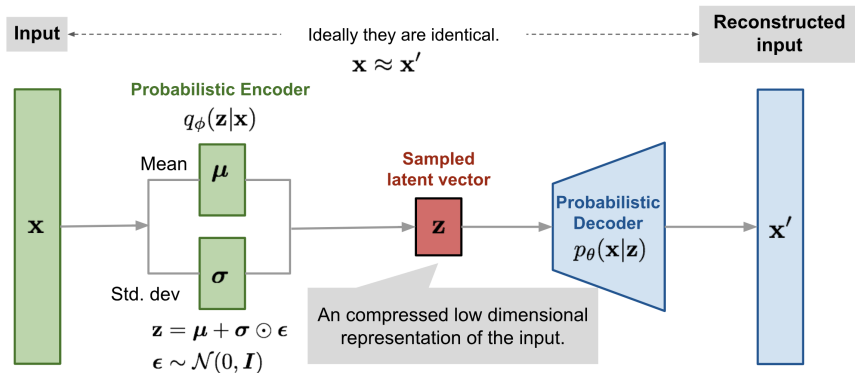
The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,
- third, the sampled point is decoded and the reconstruction error can be computed,
- finally, the reconstruction error is backpropagated through the network.

Assumptions

- In practice, the encoded distributions are chosen to be normal so that the encoder can be trained to return the mean and the covariance matrix that describe these Gaussians.
- The distributions returned by the encoder are enforced to be close to a standard normal distribution.

Variational Autoencoders



Variational Autoencoders

Let f is our encoder, g is the decoder and D is our training dataset. In this case we will minimize the following loss function

$$\sum_{x \in D} L(x, g(f(x))) + \lambda KL(N(\mu, \Sigma) || N(0, I)).$$